

Practica-6-modulo3-postgresql-18-administracion

vamos a crear la aplicacion completa con postgres 18 crando un esquema que se llame ventas. un rol usuario. y un rol con login para hacer pedidos. Una tabla clientes con un campo que se llame region.(US,EU). los usuarios solo podran listar y insertar registros cuando la region sea igual a la de una variable llamada app_region. por la tanto hay que hacer rls y una politica de seguridad. tambien hay que hacer un trigger para que actualize el stock. y si ves algo que que se me pasa lo agregas tambien. todo paso por paso y explicado.

Diseñaremos la arquitectura completa en PostgreSQL 18 cubriendo todos los aspectos requeridos: creación del esquema, roles con distintas jerarquías de permisos (administrador vs. usuario de aplicación), Row-Level Security (RLS) basado en variables de sesión (`app_region`), tablas relacionadas, trigger de validación/descuento de stock y funciones helper.

Arquitectura General del Ejemplo

1. Esquema: ventas

2. Roles:

Medium

- `rol_usuario_base`: Rol sin login para agrupación de permisos.
- `app_ventas_user`: Rol con LOGIN que usará la aplicación para conectarse.

3. Tablas:

- `ventas.clientes`: Con columna `region` (US o EU). Posee políticas RLS para restringir SELECT e INSERT.

Medium

- `ventas.productos`: Catálogo de inventario con control de stock.
- `ventas.pedidos`: Histórico de compras con FK hacia `clientes` y `productos`.

4. Seguridad / RLS: Filtro activo contra la variable de sesión `app_region`.

Bytebase

5. Automatización: Trigger que liquida stock en `productos` previa verificación de disponibilidad.

Paso 1: Esquema y Configuración de Roles

Creamos el esquema `ventas`, el rol genérico para permisos y el usuario de la aplicación.

SQL

```
-- 1. Crear el esquema
CREATE SCHEMA IF NOT EXISTS ventas;
```

```
-- 2. Crear un rol de grupo (sin login) para administrar permisos de manera
limpia
CREATE ROLE rol_usuario_base NOLOGIN;

-- 3. Crear el rol con LOGIN que utilizará el backend de la aplicación
CREATE ROLE app_ventas_user WITH LOGIN PASSWORD 'PasswordSegura123!';

-- Asignar el rol base al usuario de login
GRANT rol_usuario_base TO app_ventas_user;

-- Otorgar permisos de uso sobre el esquema
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
```

Paso 2: Creación de las Tablas y Constraints

Creamos las tres tablas en el esquema `ventas`. Añadimos restricciones `CHECK` para validar la región y que el stock/precios sean válidos.

SQL

```
-- Tabla de Clientes
CREATE TABLE ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla de Productos
CREATE TABLE ventas.productos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL CHECK (precio >= 0),
    stock INT NOT NULL CHECK (stock >= 0)
);

-- Tabla de Pedidos
CREATE TABLE ventas.pedidos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id INT NOT NULL REFERENCES ventas.clientes(id) ON DELETE RESTRICT,
    producto_id INT NOT NULL REFERENCES ventas.productos(id) ON DELETE RESTRICT,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Paso 3: Asignación de Permisos de Tablas y Secuencias

Asignamos permisos sobre las tablas al `rol_usuario_base`.

SQL

```
-- Permisos de lectura/escritura en las tablas
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA ventas TO rol_usuario_base;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA ventas TO rol_usuario_base;
```

Paso 4: Implementación de Row-Level Security (RLS)

Configuramos RLS en `ventas.clientes`. La consulta obtendrá dinámicamente la variable personalizada `app.app_region` mediante la función de PostgreSQL `current_setting('app.app_region', true)`.

SQL

```
-- 1. Habilitar RLS en la tabla clientes
ALTER TABLE ventas.clientes ENABLE ROW LEVEL SECURITY;

-- 2. Política para SELECT: Solo listar clientes cuya región coincida con
app.app_region
CREATE POLICY policy_clientes_select ON ventas.clientes
    FOR SELECT
    TO rol_usuario_base
    USING (region = current_setting('app.app_region', true));

-- 3. Política para INSERT: Solo permitir insertar clientes cuya región sea
app.app_region
CREATE POLICY policy_clientes_insert ON ventas.clientes
    FOR INSERT
    TO rol_usuario_base
    WITH CHECK (region = current_setting('app.app_region', true));
Nota importante:
```

- USING aplica para el filtrado de filas existentes (SELECT).
- WITH CHECK aplica a las filas que entran a la base de datos (INSERT / UPDATE).
- El segundo parámetro true en current_setting(..., true) evita que lance un error crítico si la variable aún no fue declarada en la sesión (retornará NULL).

Paso 5: Trigger para Actualización Automática de Stock

Creamos la función en PL/pgSQL y la asociamos como BEFORE INSERT en ventas.pedidos.

1. Función con SECURITY DEFINER y Bloqueo Selectivo

Usamos SECURITY DEFINER para asegurar que el usuario ejecutor pueda modificar el stock aunque RLS o permisos directos estén limitados, y aplicamos FOR UPDATE para evitar problemas de concurrencia (Race Conditions).

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()
RETURNS TRIGGER
SECURITY DEFINER
AS $$
DECLARE
    v_stock_actual INT;
BEGIN
    -- Bloquear y obtener el stock actual del producto para evitar race
conditions en peticiones simultáneas
    SELECT stock INTO v_stock_actual
    FROM ventas.productos
    WHERE id = NEW.producto_id
    FOR UPDATE;

    -- Validar existencia del producto
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El producto con ID % no existe.', NEW.producto_id;
    END IF;

    -- Validar disponibilidad
    IF v_stock_actual < NEW.cantidad THEN
```

```

        RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:
%, Solicitado: %',
        NEW.producto_id, v_stock_actual, NEW.cantidad;
    END IF;

    -- Actualizar stock
    UPDATE ventas.productos
    SET stock = stock - NEW.cantidad
    WHERE id = NEW.producto_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

2. Declaración del Trigger

SQL

```

CREATE TRIGGER tr_actualizar_stock_pedido
BEFORE INSERT ON ventas.pedidos
FOR EACH ROW
EXECUTE FUNCTION ventas.fn_descontar_stock_pedido();

```

Paso 6: Demostración Completa de Uso y Verificación

A continuación, simula el comportamiento de la aplicación conectándose con el rol `app_ventas_user` y ajustando la variable de región.

1. Cargar datos iniciales (Ejecutar como Admin / Superusuario)

SQL

```

-- Insertar productos en inventario
INSERT INTO ventas.productos (nombre, precio, stock) VALUES
('Laptop Pro 16', 1500.00, 10),
('Teclado Mecánico', 120.00, 5);

```

2. Iniciar sesión simulada como `app_ventas_user`

SQL

```

-- Cambiamos de rol dentro de la sesión de PostgreSQL
SET ROLE app_ventas_user;

-- Intentamos listar clientes sin definir la región (Resultado: 0 filas por RLS)
SELECT * FROM ventas.clientes;

-----
-- CONFIGURACIÓN DE REGIÓN DE APLICACIÓN: US
-----
SET LOCAL app.app_region = 'US';

-- Insertar un cliente en la región US (Permitido)
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');

-- Intentar insertar un cliente de la región EU estando en región US (Bloqueado
por RLS)
-- ERROR: new row violates row-level security policy for table "clientes"
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');

-- Consultar clientes (Solo devolverá registros de 'US')
SELECT * FROM ventas.clientes;

```

```

-----
-- REALIZAR UN PEDIDO Y VERIFICAR STOCK
-----
-- Insertar un pedido de 3 Laptops (Producto ID 1) para John Doe (Cliente ID 1)
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 1, 3);

-- Consultar el stock actualizado de la Laptop (Bajó de 10 a 7)
SELECT * FROM ventas.productos WHERE id = 1;

-- Intentar hacer un pedido que excede el stock (Producto ID 2 tiene 5 unidades)
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 2, 10);
-- ERROR: Stock insuficiente para el producto ID 2. Disponible: 5, Solicitado:
10

-----
-- CAMBIO DE REGIÓN EN LA APLICACIÓN: EU
-----
SET LOCAL app.app_region = 'EU';

-- Insertar cliente de la región EU
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');

-- Al consultar, solo veremos a Pierre Dubois (John Doe queda filtrado)
SELECT * FROM ventas.clientes;

```

Detalles Adicionales Incluidos

1. **SET LOCAL app.app_region:** Usar SET LOCAL asegura que la variable de región solo persista durante la transacción actual, lo cual previene fugas de contexto cuando se utilizan pools de conexiones (como PgBouncer o HikariCP).
2. **SECURITY DEFINER** en la función del Trigger: Permite descontar el inventario en la tabla productos de forma segura aunque el rol ejecutor no tenga privilegios administrativos directos.
3. **Control de Concurrencia (FOR UPDATE):** Previene race conditions (condiciones de carrera) si dos usuarios intentan pedir el último ítem en inventario exactamente al mismo tiempo.

Bytebase

¿Quieres ver cómo integrar esta configuración con un pool de conexiones en Node.js, Python o Java?

Sí

otra política para que no se puedan hacer pedidos si no coincide la región

Para asegurarte de que un cliente solo pueda realizar pedidos en su propia región (es decir, un usuario conectado con `app.app_region = 'US'` no pueda registrar un pedido usando el `cliente_id` de un cliente de la región 'EU'), tenemos dos formas principales de implementarlo en PostgreSQL:

1. Mediante Row-Level Security (RLS) en la tabla **pedidos** (A nivel de políticas de seguridad SQL).
2. Dentro del Trigger de la tabla **pedidos** (A nivel de lógica de negocio en la base de datos).

A continuación te muestro ambas opciones explicadas paso a paso.

Opción 1: Mediante Política RLS en ventas.pedidos (Recomendada)

Podemos activar RLS en la tabla `ventas.pedidos` y crear una política que valide, mediante un `EXISTS`, que el `cliente_id` del pedido pertenece a la región configurada en `app.app_region`.

1. Habilitar RLS en ventas.pedidos

SQL

```
ALTER TABLE ventas.pedidos ENABLE ROW LEVEL SECURITY;
```

2. Crear la Política de INSERT

SQL

```
CREATE POLICY policy_pedidos_insert ON ventas.pedidos
FOR INSERT
TO rol_usuario_base
WITH CHECK (
    EXISTS (
        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
              AND c.region = current_setting('app.app_region', true)
    )
);
```

3. Crear la Política de SELECT (Opcional pero recomendada)

Para que los usuarios tampoco puedan listar los pedidos de clientes de otras regiones:

SQL

```
CREATE POLICY policy_pedidos_select ON ventas.pedidos
FOR SELECT
TO rol_usuario_base
USING (
    EXISTS (
        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
              AND c.region = current_setting('app.app_region', true)
    )
);
```

Opción 2: Validar la Región dentro del Trigger de Pedidos

Si prefieres centralizar la lógica de negocio y dar un mensaje de error más explícito cuando la región no coincida, podemos modificar la función que ya teníamos en el trigger `fn_descontar_stock_pedido`:

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()
RETURNS TRIGGER
SECURITY DEFINER
AS $$
DECLARE
    v_stock_actual INT;
    v_region_cliente VARCHAR(2);
    v_app_region VARCHAR(2);
BEGIN
```

```

-- 1. Obtener la región configurada en la sesión actual
v_app_region := current_setting('app.app_region', true);

-- 2. Obtener la región del cliente asociado al pedido
SELECT region INTO v_region_cliente
FROM ventas.clientes
WHERE id = NEW.cliente_id;

-- Validar si el cliente existe
IF NOT FOUND THEN
    RAISE EXCEPTION 'El cliente con ID % no existe.', NEW.cliente_id;
END IF;

-- 3. VALIDACIÓN DE REGIÓN: Si la región del cliente no coincide con
app.app_region, rechazar el pedido
IF v_region_cliente IS DISTINCT FROM v_app_region THEN
    RAISE EXCEPTION 'Operación no permitida: El cliente ID % pertenece a la
región %, pero la sesión actual está configurada en %.',
NEW.cliente_id, v_region_cliente, COALESCE(v_app_region, 'SIN
CONFIGURAR');
END IF;

-- 4. Bloquear y verificar stock del producto
SELECT stock INTO v_stock_actual
FROM ventas.productos
WHERE id = NEW.producto_id
FOR UPDATE;

IF NOT FOUND THEN
    RAISE EXCEPTION 'El producto con ID % no existe.', NEW.producto_id;
END IF;

IF v_stock_actual < NEW.cantidad THEN
    RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:
%, Solicitado: %',
NEW.producto_id, v_stock_actual, NEW.cantidad;
END IF;

-- 5. Descontar stock
UPDATE ventas.productos
SET stock = stock - NEW.cantidad
WHERE id = NEW.producto_id;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Prueba de Funcionamiento

Simulemos el escenario donde intentamos crear un pedido para un cliente de la región 'EU' cuando la app está configurada en la región 'US':

SQL

```

-- Cambiamos al usuario de la aplicación
SET ROLE app_ventas_user;

-- Caso de uso:
-- Supongamos que en la BD existe:
-- Cliente ID 1 (John Doe) -> Región 'US'
-- Cliente ID 2 (Pierre Dubois) -> Región 'EU'

```

-- CONFIGURACIÓN DE REGIÓN EN LA SESIÓN: US

```

-----
SET LOCAL app.app_region = 'US';

-- 1. Intentar hacer un pedido para John Doe (Cliente US - ID 1) -> PERMITIDO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (1, 1, 1);

-- 2. Intentar hacer un pedido para Pierre Dubois (Cliente EU - ID 2) estando en
sesión US -> BLOQUEADO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad)
VALUES (2, 1, 1);

-- Si usas Opción 1 (RLS), responderá:
-- ERROR: new row violates row-level security policy for table "pedidos"

-- Si usas Opción 2 (Trigger), responderá:
-- ERROR: Operación no permitida: El cliente ID 2 pertenece a la región EU, pero
la sesión actual está configurada en US.

```

¿Cuál opción elegir?

- Opción 1 (RLS): Es la mejor práctica estándar en PostgreSQL si quieres mantener las políticas de seguridad nativas e invisibles para la lógica de la aplicación.
- Opción 2 (Trigger): Es preferible si deseas controlar los mensajes de error personalizados para que el backend reconozca exactamente por qué falló la inserción.

¿Quieres que añadamos políticas RLS para UPDATE y DELETE en las tablas de pedidos y clientes?

Sí

ahora creame un listado de consultas para poder ver todo por sql. roles, esquemas, politicas, trigger, tablas por esquema, permisos etc.

Aquí tienes un listado completo de consultas SQL nativas (utilizando el catálogo del sistema de PostgreSQL `pg_catalog` e `information_schema`) para auditar e inspeccionar roles, esquemas, tablas, políticas de RLS, triggers y permisos.

1. Consultas de Roles y Usuarios

SQL

```

-- Ver todos los roles, si tienen LOGIN, si son superusuarios y sus miembros
SELECT
    r.rolname AS nombre_rol,
    r.rolcanlogin AS permite_login,
    r.rolsuper AS es_superuser,
    ARRAY(
        SELECT m.rolname
        FROM pg_roles m
        JOIN pg_auth_members am ON m.oid = am.roleid
        WHERE am.member = r.oid
    ) AS pertenece_a_grupos
FROM pg_roles r
ORDER BY r.rolname;

```

2. Consultas de Esquemas

SQL


```
-- Listar todos los esquemas creados (excluyendo esquemas del sistema)
SELECT
    schema_name AS esquema,
    schema_owner AS propietario
FROM information_schema.schemata
WHERE schema_name NOT IN ('pg_catalog', 'information_schema')
    AND schema_name NOT LIKE 'pg_temp%'
ORDER BY schema_name;
```

3. Consultas de Tablas por Esquema

SQL

```
-- Listar todas las tablas indicando esquema, nombre y si tienen RLS habilitado
SELECT
    schemaname AS esquema,
    tablename AS tabla,
    tableowner AS propietario,
    rowsecurity AS rls_habilitado,
    forcerowsecurity AS rls_forzado
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY schemaname, tablename;
```

4. Consultas de Políticas de Seguridad (RLS)

SQL

```
-- Ver todas las políticas de Row-Level Security definidas en las tablas
SELECT
    schemaname AS esquema,
    tablename AS tabla,
    policyname AS nombre_politica,
    roles AS roles_afectados,
    cmd AS operacion, -- SELECT, INSERT, UPDATE, DELETE o ALL
    qual AS condicion_using,
    with_check AS condicion_with_check
FROM pg_policies
WHERE schemaname = 'ventas' -- Puedes quitar el WHERE para ver todos los
esquemas
ORDER BY schemaname, tablename, policyname;
```

5. Consultas de Triggers y Funciones

SQL

```
-- Listar todos los triggers configurados, su tabla y la función que ejecutan
SELECT
    event_object_schema AS esquema,
    event_object_table AS tabla,
    trigger_name AS nombre_trigger,
    action_timing AS momento, -- BEFORE, AFTER, INSTEAD OF
    event_manipulation AS evento, -- INSERT, UPDATE, DELETE
    action_statement AS funcion_ejecutada
FROM information_schema.triggers
WHERE event_object_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY event_object_schema, event_object_table;

-- Ver el código fuente de las funciones personalizadas (como la del trigger)
SELECT
    n.nspname AS esquema,
    p.proname AS nombre_funcion,
    pg_get_functiondef(p.oid) AS codigo_fuente
FROM pg_proc p
JOIN pg_namespace n ON n.oid = p.pronamespace
```

```
WHERE n.nspname = 'ventas';
```

6. Consultas de Permisos (ACLs) sobre Tablas

SQL

```
-- Ver los permisos explícitos otorgados a roles sobre las tablas de un esquema
SELECT
    grantee AS rol_otorgado,
    table_schema AS esquema,
    table_name AS tabla,
    string_agg(privilege_type, ', ') AS permisos
FROM information_schema.role_table_grants
WHERE table_schema = 'ventas'
GROUP BY grantee, table_schema, table_name
ORDER BY table_schema, table_name, grantee;
```

7. Consultas de Claves Foráneas y Relaciones

SQL

```
-- Ver las relaciones entre tablas (Foreign Keys)
SELECT
    tc.table_schema AS esquema,
    tc.table_name AS tabla_origen,
    kcu.column_name AS columna_origen,
    ccu.table_name AS tabla_destino,
    ccu.column_name AS columna_destino,
    tc.constraint_name AS nombre_restriccion
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
    AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
    AND ccu.table_schema = tc.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
    AND tc.table_schema = 'ventas';
```

Bonus: Atajos en la consola interactiva `psql`

Si estás utilizando la terminal con el comando `psql`, estos comandos aceleran la inspección sin necesidad de escribir consultas complejas:

- `\du` : Listar roles y usuarios.
- `\dn` : Listar esquemas.
- `\dt ventas.*` : Listar tablas del esquema `ventas`.
- `\d+ ventas.clientes` : Ver la estructura completa de la tabla `clientes` (columnas, FK, triggers y políticas RLS).
- `\df+ ventas.*` : Listar funciones del esquema `ventas` y ver su código.

¿Quieres una consulta para auditar las conexiones activas y las variables de sesión actuales?

Sí

ERROR: no existe la relación «ventas.clientes»

Este error ocurre principalmente por dos razones comunes:

1. Olvidaste ejecutar el comando `CREATE SCHEMA ventas;` o la tabla `clientes` dentro de ese esquema antes de hacer la consulta.
2. Estás ejecutando la consulta como un usuario/rol que no tiene permisos de uso (`USAGE`) sobre el esquema `ventas`, por lo que PostgreSQL actúa como si la tabla no existiera para proteger la seguridad.

Solución Paso a Paso

1. Verifica si el esquema y la tabla realmente existen

Ejecuta esta consulta con un usuario administrador/superusuario (por ejemplo, `postgres`):

SQL

```
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_name = 'clientes';
```

- Si la consulta no devuelve nada: La tabla no ha sido creada. Ejecuta primero la creación del esquema y las tablas:

SQL

```
CREATE SCHEMA IF NOT EXISTS ventas;
```

```
CREATE TABLE ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. Otorga los permisos necesarios al usuario/rol

Si la tabla sí existe pero te da ese error al ejecutar desde el rol de la aplicación (`app_ventas_user` o `rol_usuario_base`), debes otorgarle permisos explícitos sobre el esquema y la tabla desde tu usuario administrador:

SQL

```
-- 1. Dar permiso para "ver/usar" el esquema ventas
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;

-- 2. Dar permisos sobre las tablas dentro del esquema
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO
rol_usuario_base;

-- 3. Asegurar que app_ventas_user heredó el rol base
GRANT rol_usuario_base TO app_ventas_user;
```

3. Revisa tu `search_path` (Opcional)

Si prefieres escribir `SELECT * FROM clientes;` en lugar de `SELECT * FROM ventas.clientes;`, puedes agregar el esquema `ventas` a la ruta de búsqueda de la sesión:

SQL

```
SET search_path TO ventas, public;
```

igual: -- 1. Dar permiso para "ver/usar" el esquema ventas

```
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
```

-- 2. Dar permisos sobre las tablas dentro del esquema

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO rol_usuario_base;
```

-- 3. Asegurar que app_ventas_user heredó el rol base

```
GRANT rol_usuario_base TO app_ventas_user;
```

Si el error no existe la relación «ventas.clientes» persiste incluso tras haber otorgado los permisos, el problema no es de permisos, sino de dónde se creó la tabla o de la sesión activa.

Aplica estos pasos para diagnosticar y corregir el problema:

1. Comprueba si la tabla realmente se creó en esa Base de Datos

En PostgreSQL, los esquemas y tablas son locales a cada Base de Datos. Si creaste la tabla en la base de datos `postgres` e intentas hacer la consulta conectado a otra base de datos (por ejemplo `ventas_db`), devolverá exactamente ese error.

Ejecuta esta consulta con tu usuario administrador para ver si la tabla existe en algún esquema:

SQL

```
SELECT table_catalog AS base_de_datos, table_schema, table_name
FROM information_schema.tables
WHERE table_name = 'clientes';
```

- Si la consulta devuelve 0 filas: La tabla nunca llegó a ejecutarse en la base de datos actual.
- Si la consulta muestra `public` en `table_schema`: La creaste en el esquema público por defecto sin el prefijo `ventas..`

2. Script Completo de Inicialización (Limpio)

Para asegurar que todo se cree en el orden correcto y en la base de datos adecuada, ejecuta todo este bloque desde la consola de administración (como usuario `postgres` o `superusuario`):

SQL

```
-- 1. Crear el esquema
CREATE SCHEMA IF NOT EXISTS ventas;

-- 2. Crear los roles si no existen
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname =
'rol_usuario_base') THEN
        CREATE ROLE rol_usuario_base NOLOGIN;
```

```

END IF;

IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname =
'app_ventas_user') THEN
    CREATE ROLE app_ventas_user WITH LOGIN PASSWORD 'PasswordSegura123!';
END IF;
END
$$;

-- 3. Crear las tablas dentro del esquema ventas
CREATE TABLE IF NOT EXISTS ventas.clientes (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    region VARCHAR(2) NOT NULL CHECK (region IN ('US', 'EU')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ventas.productos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10, 2) NOT NULL CHECK (precio >= 0),
    stock INT NOT NULL CHECK (stock >= 0)
);

CREATE TABLE IF NOT EXISTS ventas.pedidos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id INT NOT NULL REFERENCES ventas.clientes(id) ON DELETE RESTRICT,
    producto_id INT NOT NULL REFERENCES ventas.productos(id) ON DELETE RESTRICT,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Asignar permisos sobre el esquema, tablas y secuencias
GRANT USAGE ON SCHEMA ventas TO rol_usuario_base;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ventas TO
rol_usuario_base;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA ventas TO rol_usuario_base;
GRANT rol_usuario_base TO app_ventas_user;

```

3. Asignar permisos por defecto para futuras tablas

Si la tabla `clientes` fue creada por el usuario `postgres` después de otorgar el `GRANT`, las versiones de PostgreSQL previas o ciertas configuraciones no aplican retroactivamente el permiso a tablas nuevas a menos que se use `ALTER DEFAULT PRIVILEGES`:

SQL

```

ALTER DEFAULT PRIVILEGES IN SCHEMA ventas
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO rol_usuario_base;

ALTER DEFAULT PRIVILEGES IN SCHEMA ventas
GRANT USAGE, SELECT ON SEQUENCES TO rol_usuario_base;

```

4. Prueba de Verificación

Una vez ejecutado el script anterior, prueba la consulta cambiando al rol de aplicación:

SQL

```

-- Cambiar temporalmente de rol para probar
SET ROLE app_ventas_user;

-- Probar consulta explícita apuntando al esquema

```

```
SELECT * FROM ventas.clientes;
```

Si ejecutas `SELECT * FROM ventas.clientes;` tras aplicar el bloque anterior y continúa fallando, confirma con el comando `\C` (en psql) o en los detalles de tu cliente gráfico (DBeaver, pgAdmin) el nombre de la base de datos activa a la que estás conectado.

```
SELECT * FROM ventas.clientes; y no me devuelve nada
```

Si no devuelve ningún error y simplemente te muestra 0 filas (vacío), el comportamiento es exactamente el que diseñamos por Row-Level Security (RLS).

Cuando ejecutamos RLS en la tabla `ventas.clientes`, definimos que solo se muestren los registros donde la columna `region` coincida con la variable de sesión `app.app_region`. Si esa variable no se ha definido en la sesión actual, vale `NULL`, por lo que PostgreSQL oculta todas las filas por seguridad.

Diagnóstico Rápido

Ejecuta estas consultas en orden para entender qué está pasando:

1. Comprobar cuántos registros hay en realidad (como Administrador)

Si cambias a un rol que ignore RLS (como `postgres`), verás si hay datos insertados:

SQL

```
-- Restablecer al rol administrador de la sesión
RESET ROLE;
```

```
-- Consultar la tabla sin filtros de RLS
SELECT * FROM ventas.clientes;
```

2. Consultar como `app_ventas_user` definiendo la variable de región

Si estás conectado con el rol de la aplicación o usas `SET ROLE app_ventas_user;`, debes declarar la región antes de hacer la consulta:

SQL

```
SET ROLE app_ventas_user;
```

```
-- 1. Definir la región para esta sesión/transacción
SET LOCAL app.app_region = 'US';
```

```
-- 2. Ahora la consulta devolverá los clientes de esa región
SELECT * FROM ventas.clientes;
```

¿Por qué sucede esto?

Recuerda la política RLS que creamos:

SQL

```
CREATE POLICY policy_clientes_select ON ventas.clientes
  FOR SELECT
  TO rol_usuario_base
  USING (region = current_setting('app.app_region', true));
```

- `current_setting('app.app_region', true)` evalúa a `NULL` si no has ejecutado `SET app.app_region = 'US';` antes.
- La condición `region = NULL` en SQL resulta en `UNKNOWN` (falso), por lo que PostgreSQL filtra el 100% de las filas.

Flujo Completo de Prueba (Copia y Ejecuta)

Para probar insertar y consultar datos correctamente con RLS habilitado:

SQL

```
-- 1. Iniciar sesión como usuario de aplicación
SET ROLE app_ventas_user;

-- 2. Definir que la app está operando en la región 'US'
SET LOCAL app.app_region = 'US';

-- 3. Insertar un cliente (Solo funcionará si la región enviada coincide con 'US')
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');

-- 4. Consultar (Ahora devolverá a John Doe)
SELECT * FROM ventas.clientes;

-- 5. Cambiar la región a 'EU'
SET LOCAL app.app_region = 'EU';

-- 6. Al consultar ahora, volverá a salir 0 filas porque John Doe es de 'US'
SELECT * FROM ventas.clientes;
¿Quieres que ajustemos la política RLS para que muestre todos los registros si la variable
app_region no está definida?
```

Sí

he entrado como: `psql -h 127.0.0.1 -U app_ventas_user -d appdb`

Exacto, ese comportamiento es 100% normal y esperado con el diseño de Row-Level Security (RLS) que configuramos.

Como has iniciado sesión directamente con el usuario de la aplicación `app_ventas_user`, PostgreSQL le aplica las políticas RLS estrictamente. En este momento, la variable de sesión `app.app_region` aún no tiene ningún valor asignado (es `NULL`), por lo que la base de datos oculta todas las filas por seguridad.

Cómo trabajar desde esa terminal (psql)

Estando dentro de tu sesión actual con `app_ventas_user`, ejecuta los siguientes comandos paso a paso:

1. Establece la región en la sesión

SQL

```
SET app.app_region = 'US';
```

2. Inserta un cliente para esa región

SQL

```
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('John Doe', 'john@example.com', 'US');
```

3. Realiza la consulta nuevamente

SQL

```
SELECT * FROM ventas.clientes;
Ahora sí verás la fila de John Doe.
```

4. Intenta insertar un cliente de otra región (Prueba de Seguridad)

SQL

```
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');
Lanzará un error de violación de política RLS porque tu sesión está fijada en 'US'.
```

5. Cambia de región a 'EU'

SQL

```
SET app.app_region = 'EU';

-- Insertar ahora a Pierre (Permitido)
INSERT INTO ventas.clientes (nombre, email, region)
VALUES ('Pierre Dubois', 'pierre@example.fr', 'EU');

-- Consultar
SELECT * FROM ventas.clientes;
Verás únicamente a Pierre Dubois, mientras que John Doe quedará oculto.
```

Nota sobre SET vs SET LOCAL en psql:

- En la consola interactiva psql, usa SET app.app_region = 'US'; para que el valor se mantenga durante toda tu sesión en la terminal.
- En el backend de una aplicación (Node.js, Python, Java, etc.), se utiliza SET LOCAL app.app_region = 'US'; dentro de un bloque de transacción (BEGIN ... COMMIT) para evitar que la variable contamine otras peticiones en el pool de conexiones.

```
appdb=> SET app.app_region = 'US';
```

SET

```
appdb=> INSERT INTO ventas.clientes (nombre, email, region)
```

```
VALUES ('John Doe', 'john@example.com', 'US');
```

```
INSERT 0 1
```

```
appdb=> SELECT * FROM ventas.clientes;
```

```
id | nombre | email | region | fecha_registro
```

```
----+-----+-----+-----+-----
```


(0 filas)

El hecho de que el `INSERT` haya sido exitoso (`INSERT 0 1`) pero el `SELECT` devuelva 0 filas nos indica exactamente qué está sucediendo: el `INSERT` funcionó gracias a la política con `WITH CHECK`, pero el `SELECT` está fallando porque la política con `USING` no está evaluando `current_setting('app.app_region', true)` como esperábamos.

En PostgreSQL, `current_setting('app.app_region', true)` a veces requiere que la variable haya sido registrada formalmente o que el tipo de dato coincida de manera explícita (`::text`).

Solución en 2 Pasos

Paso 1: Recrear la política de `SELECT` con casteo explícito

Conéctate como superusuario (o con el rol creador) y actualiza la política para forzar el tipo de texto y manejar posibles espacios o comillas:

SQL

```
-- Eliminar la política anterior de SELECT
DROP POLICY IF EXISTS policy_clientes_select ON ventas.clientes;

-- Volver a crearla con casteo explícito a TEXT
CREATE POLICY policy_clientes_select ON ventas.clientes
    FOR SELECT
    TO rol_usuario_base
    USING (region::text = NULLIF(current_setting('app.app_region', true),
'')::text);
```

Y haz lo mismo con la política de `INSERT` para mantener consistencia:

SQL

```
DROP POLICY IF EXISTS policy_clientes_insert ON ventas.clientes;

CREATE POLICY policy_clientes_insert ON ventas.clientes
    FOR INSERT
    TO rol_usuario_base
    WITH CHECK (region::text = NULLIF(current_setting('app.app_region', true),
'')::text);
```

Paso 2: Probar desde tu sesión `app_ventas_user`

Regresa a la terminal donde estás conectado como `app_ventas_user` y ejecuta:

SQL

```
-- 1. Asignar la variable
SET app.app_region = 'US';

-- 2. Verificar qué valor está leyendo PostgreSQL exactamente
SELECT current_setting('app.app_region', true);

-- 3. Realizar el SELECT
SELECT * FROM ventas.clientes;
```

¿Por qué ocurrió esto?

1. En la política de INSERT, PostgreSQL evalúa la condición al momento de la inserción.
2. En la política de SELECT, si no hay una conversión explícita entre el tipo VARCHAR(2) de la columna region y el tipo devuelto por current_setting() (que es text), la comparación region = current_setting(...) puede devolver UNKNOWN o evaluar en falso según el contexto de ejecución.

Al agregar ::text y NULLIF(..., ''), aseguramos una comparación exacta entre strings.

como se asignan estas variables : 'Stock insuficiente para el producto ID %. Disponible: %, Solicitado: %',

En PostgreSQL (específicamente en PL/pgSQL), los comodines % en la orden RAISE EXCEPTION (o RAISE NOTICE, RAISE WARNING) funcionan como marcadores de posición que se reemplazan en orden por las variables o expresiones que pones separadas por comas al final de la sentencia.

Sintaxis y Ejemplo

La sintaxis básica es:

SQL

```
RAISE EXCEPTION 'mensaje con comodines %', variable1, variable2, variable3;
```

Para tu caso concreto, la asignación se hace pasando las variables exactamente en el mismo orden en el que aparecen los %:

SQL

```
RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible: %, Solicitado: %',  
    NEW.producto_id,  
    v_stock_actual,  
    NEW.cantidad;
```

¿Cómo se mapean?

1. El primer % se reemplaza por NEW.producto_id
2. El segundo % se reemplaza por v_stock_actual
3. El tercer % se reemplaza por NEW.cantidad

Ejemplo dentro de la función PL/pgSQL

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_descontar_stock_pedido()  
RETURNS TRIGGER AS $$  
DECLARE  
    v_stock_actual INT := 5; -- Ejemplo: hay 5 unidades  
BEGIN  
    -- Supongamos que NEW.producto_id = 12 y NEW.cantidad = 10
```

```

        IF v_stock_actual < NEW.cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente para el producto ID %. Disponible:
%, Solicitado: %',
                NEW.producto_id,
                v_stock_actual,
                NEW.cantidad;
        END IF;

        RETURN NEW;
END;
$$ LANGUAGE plpgsql;
Salida generada al intentar insertar:

```

```

        ERROR: Stock insuficiente para el producto ID 12.
        Disponible: 5, Solicitado: 10

```

Tip Adicional: Formato de Tipos

Si necesitas dar formato específico a un valor dentro del mensaje, puedes usar modificadores de formato con `format()`:

- `%o %s`: Imprime el valor como texto genérico (comportamiento por defecto de `RAISE`).
- `%I`: Se usa para formatear identificadores (nombres de tablas/columnas de forma segura).
- `%L`: Se usa para literales (añade comillas simples automáticamente y gestiona valores `NULL`).

en las nuevas políticas de pedidos habría que hacer también una conversión con la variable `region`?

Sí, es una excelente idea aplicarle la misma conversión `::text` a la variable en las políticas de pedidos por consistencia y para prevenir el mismo comportamiento que experimentaste con la tabla `clientes`.

Al realizar la comparación dentro de un subcomando `EXISTS`, PostgreSQL compara la columna `c.region` contra el resultado de `current_setting(...)`. Usar la misma función limpiadora `NULLIF(current_setting('app.app_region', true), '')::text` garantiza que el valor de la variable se interprete siempre como un `text` limpio.

Recreación de las Políticas para `ventas.pedidos`

Conéctate como superusuario (o rol propietario) y actualiza las políticas de la tabla `pedidos`:

SQL

```

-- 1. Asegurar que RLS esté activo en la tabla
ALTER TABLE ventas.pedidos ENABLE ROW LEVEL SECURITY;

-- 2. Eliminar políticas previas si existen
DROP POLICY IF EXISTS policy_pedidos_insert ON ventas.pedidos;
DROP POLICY IF EXISTS policy_pedidos_select ON ventas.pedidos;

-- 3. Política para INSERT: Validar que el cliente pertenece a la región activa
CREATE POLICY policy_pedidos_insert ON ventas.pedidos
    FOR INSERT
    TO rol_usuario_base
    WITH CHECK (
        EXISTS (

```

```

        SELECT 1
        FROM ventas.clientes c
        WHERE c.id = NEW.cliente_id
              AND c.region::text = NULLIF(current_setting('app.app_region',
true), '')::text
      )
    );

-- 4. Política para SELECT: Permitir listar solo pedidos de la región activa
CREATE POLICY policy_pedidos_select ON ventas.pedidos
  FOR SELECT
  TO rol_usuario_base
  USING (
    EXISTS (
      SELECT 1
      FROM ventas.clientes c
      WHERE c.id = NEW.cliente_id
            AND c.region::text = NULLIF(current_setting('app.app_region',
true), '')::text
    )
  );

```

Prueba rápida en la terminal (psql) con app_ventas_user

SQL

```

-- Configurar la sesión en región US
SET app.app_region = 'US';

-- Supongamos que John Doe (ID 1) es de 'US' y Pierre Dubois (ID 2) es de 'EU'

-- 1. Hacer pedido para John Doe (ID 1) estando en 'US' -> PERMITIDO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad) VALUES (1, 1, 2);

-- 2. Intentar hacer pedido para Pierre Dubois (ID 2) estando en 'US' ->
BLOQUEADO
INSERT INTO ventas.pedidos (cliente_id, producto_id, cantidad) VALUES (2, 1, 1);
-- Devuelve: ERROR: new row violates row-level security policy for table
"pedidos"

-- 3. Consultar los pedidos de la región 'US'
SELECT * FROM ventas.pedidos;

¿Quieres agregar también políticas RLS para UPDATE y DELETE en pedidos?

```

Funciones y Procedimientos

En esta guía práctica aprenderás a crear **Funciones y Procedimientos Almacenados** en PostgreSQL 18 utilizando el esquema ventas que hemos venido construyendo.

1. La Diferencia Clave: ¿Función o Procedimiento?

Característica	Función (CREATE FUNCTION)	Procedimiento (CREATE PROCEDURE)
Valor de retorno	Obligatorio. Siempre devuelve algo (un tipo básico, una tabla o un TRIGGER).	No devuelve valor directamente. Se usa para ejecutar acciones.
Control de Transacciones	No permite COMMIT ni ROLLBACK dentro de su cuerpo. Es atómica.	Sí permite COMMIT y ROLLBACK internos (gestión manual de transacciones).
Forma de invocación	Se llama con SELECT: SELECT funcion();	Se llama con CALL: CALL procedimiento();

2. Creando Funciones (CREATE FUNCTION)

Las funciones son ideales para **calcular valores**, **consultar datos procesados** o **transformar información**.

Ejemplo 1: Función escalar simple (Calcular el Total de un Pedido)

Esta función recibe el precio de un producto y la cantidad solicitada, aplicando un descuento opcional.

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_calcular_total_pedido(  
    p_precio NUMERIC,  
    p_cantidad INT,  
    p_descuento_porcentaje NUMERIC DEFAULT 0  
)  
RETURNS NUMERIC AS $$  
DECLARE  
    v_subtotal NUMERIC;  
    v_total NUMERIC;  
BEGIN  
    -- 1. Operación básica  
    v_subtotal := p_precio * p_cantidad;  
  
    -- 2. Aplicar descuento  
    v_total := v_subtotal - (v_subtotal * (p_descuento_porcentaje / 100));
```

```
-- 3. Retornar el resultado
RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Cómo probar la función:

SQL

```
-- Sin descuento (usa el valor por defecto 0)
SELECT ventas.fn_calcular_total_pedido(150.00, 2);
-- Devuelve: 300.00
```

```
-- Con un 10% de descuento
SELECT ventas.fn_calcular_total_pedido(150.00, 2, 10);
-- Devuelve: 270.00
```

Ejemplo 2: Función que devuelve una Tabla (RETURNS TABLE)

Imagina que quieres obtener un resumen de ventas de un cliente específico en una sola consulta estructurada.

SQL

```
CREATE OR REPLACE FUNCTION ventas.fn_resumen_compras_cliente(p_cliente_id INT)
RETURNS TABLE (
    pedido_id INT,
    producto_nombre VARCHAR,
    cantidad INT,
    fecha_pedido TIMESTAMP
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.id,
        prod.nombre,
        p.cantidad,
        p.fecha
    FROM ventas.pedidos p
    JOIN ventas.productos prod ON p.producto_id = prod.id
    WHERE p.cliente_id = p_cliente_id;
END;
$$ LANGUAGE plpgsql;
```

Cómo probar la función:

SQL

```
SELECT * FROM ventas.fn_resumen_compras_cliente(1);
```

3. Creando Procedimientos Almacenados (CREATE PROCEDURE)

Los procedimientos almacenados se utilizan para **ejecutar flujos de trabajo complejas** (procesar datos, insertar en múltiples tablas, reponer stock masivo)

donde no necesitas una respuesta inmediata en filas, sino realizar cambios en el estado de la base de datos.

Ejemplo 3: Procedimiento de Reabastecimiento de Inventario

Crearemos un procedimiento para reponer el stock de un producto y registrar la acción en una tabla de auditoría.

Paso previo: Crear la tabla de auditoría

SQL

```
CREATE TABLE IF NOT EXISTS ventas.historial_reabastecimiento (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    producto_id INT REFERENCES ventas.productos(id),  
    cantidad_agregada INT NOT NULL,  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Creación del Procedimiento:

SQL

```
CREATE OR REPLACE PROCEDURE ventas.pr_reabastecer_producto(  
    p_producto_id INT,  
    p_cantidad INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    -- Validar que la cantidad a reponer sea lógica  
    IF p_cantidad <= 0 THEN  
        RAISE EXCEPTION 'La cantidad a reponer debe ser mayor a 0. Solicitado: %',  
p_cantidad;  
    END IF;  
  
    -- 1. Incrementar el stock del producto  
    UPDATE ventas.productos  
    SET stock = stock + p_cantidad  
    WHERE id = p_producto_id;  
  
    -- Validar si el producto realmente existía  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'No se encontró el producto con ID %', p_producto_id;  
    END IF;  
  
    -- 2. Registrar el movimiento en el historial  
    INSERT INTO ventas.historial_reabastecimiento (producto_id, cantidad_agregada)  
    VALUES (p_producto_id, p_cantidad);  
  
    -- Confirmación explícita (opcional, PostgreSQL lo hace automáticamente al terminar la  
transacción)  
    COMMIT;  
END;  
$;
```

Cómo probar el procedimiento:

SQL

-- Ejecutar la llamada con CALL

```
CALL ventas.pr_reabastecer_producto(1, 50);
```

-- Verificar el incremento en el producto

```
SELECT id, nombre, stock FROM ventas.productos WHERE id = 1;
```

-- Verificar la entrada en el historial

```
SELECT * FROM ventas.historial_reabastecimiento;
```

4. Conceptos Clave de PL/pgSQL para recordar

1. **Variables:** Se declaran en el bloque DECLARE con su tipo (v_total NUMERIC;).
2. **Asignación:** Se utiliza el operador := para asignar valores a las variables (v_subtotal := p_precio * p_cantidad;).
3. **Estructuras de control:** Puedes usar bloques condicionales IF ... THEN ... ELSE ... END IF; o bucles LOOP / WHILE.
4. **FOUND:** Es una variable booleana especial de PL/pgSQL que se vuelve TRUE si la última sentencia SQL (UPDATE, DELETE, SELECT INTO) afectó al menos una fila.